

1271018_CleanDataOnlyFirstVisit

October 28, 2018

```
In [130]: # Importing libraries
import pandas as pd
import numpy as np
import os
import math
import operator
```

```
In [131]: pd.set_option('display.max_columns', 1000)
pd.set_option('display.width', 1000)
```

```
In [132]: os.chdir('A:\\Hackathon\\Data')
```

```
In [134]: df = pd.read_csv('ImputedCleanData.csv')
df.head(10)
```

```
Out[134]:
```

	Unnamed: 0	MEDINCSTQ	SUROPTOT	AGE	CCI	FTMDTF	CM_HTN_C	CM_ALCOHOL	PP086	M
0	0	4	17252	46	3	1	0	0	0	
1	4	2	9467	79	0	0	1	0	1	
2	6	4	24434	73	6	144	0	0	1	
3	7	4	53296	73	7	186	0	0	1	
4	8	3	60571	71	2	50	1	0	1	
5	9	4	8775	86	5	1	1	0	1	
6	10	3	60571	73	2	50	1	0	1	
7	12	4	8113	75	6	168	1	0	0	
8	13	3	8656	52	1	1	1	0	0	
9	14	3	24434	55	6	144	0	0	1	

```
In [135]: df.columns[df.dtypes == 'float64']
```

```
Out[135]: Index(['Unnamed: 88'], dtype='object')
```

```
In [136]: df.columns[df.isna().any()].tolist()
```

```
Out[136]: ['Unnamed: 88']
```

```
In [109]: # df['DISPUNIFORM'].value_counts()
```

```

In [110]: df['CM_AIDS'] = df['CM_AIDS'].fillna(0)
df['CM_ALCOHOL'] = df['CM_ALCOHOL'].fillna(0)
df['CM_DEPRESS'] = df['CM_DEPRESS'].fillna(0)
df['CM_DRUG'] = df['CM_DRUG'].fillna(0)
df['CM_LIVER'] = df['CM_LIVER'].fillna(0)
df['CM_OBESE'] = df['CM_OBESE'].fillna(0)
df['CM_PSYCH'] = df['CM_PSYCH'].fillna(0)
df['DISPUNIFORM'] = df['DISPUNIFORM'].fillna(6)
df['FEMALE'] = df['FEMALE'].fillna(1)
df['GENHOS'] = df['GENHOS'].fillna(1)
df['MEDINCSTQ'] = df['MEDINCSTQ'].fillna(4)
df['PAY1'] = df['PAY1'].fillna(3)
df['PPO86'] = df['PPO86'].fillna(1)
df['RACE'] = df['RACE'].fillna(6)

In [114]: for col in ['CM_AIDS', 'CM_ALCOHOL', 'CM_DEPRESS', 'CM_DRUG', 'CM_LIVER', 'CM_OBESE',
                    'GENHOS', 'MEDINCSTQ', 'PAY1', 'PPO86', 'RACE']:
    df[col] = df[col].astype('int64')

In [125]: df['TOTCHG_X'].describe()

Out[125]: count      16999.0
unique          4.0
top             4.0
freq           5299.0
Name: TOTCHG_X, dtype: float64

In [112]: df['TOTCHG_X'] = df.fillna(df.mean()['TOTCHG_X'])

In [141]: # df = df.drop(['Unnamed: 88'], axis=1)
df = df.drop(['Unnamed: 0'], axis=1)

In [142]: df.head()

Out[142]:
   MEDINCSTQ  SUROPTOT  AGE  CCI  FTMDTF  CM_HTN_C  CM_ALCOHOL  PPO86  MAPP12n  BDTOT
0           4    17252   46    3        1         0           0        0         2    42
1           2     9467   79    0        0         1           0        1         2    40
2           4    24434   73    6       144         0           0        1         2   209
3           4    53296   73    7       186         0           0        1         2   143
4           3    60571   71    2        50         1           0        1         2   208

In [143]: df.shape

Out[143]: (16999, 87)

In [145]: df.to_csv('1271018_CleanData.csv')

In [6]: # Defining a function which calculates euclidean distance between two data points
def euclideanDistance(data1, data2, length):
    distance = 0

```

```

    for x in range(length):
        distance += np.square(data1[x] - data2[x])
    return np.sqrt(distance)

# Defining our KNN model
def knn(trainingSet, testInstance, k):

    distances = {}
    sort = {}

    length = testInstance.shape[1]

    #### Start of STEP 3
    # Calculating euclidean distance between each row of training data and test data
    for x in range(len(trainingSet)):

        #### Start of STEP 3.1
        dist = euclideanDistance(testInstance, trainingSet.iloc[x], length)

        distances[x] = dist[0]
        #### End of STEP 3.1

    #### Start of STEP 3.2
    # Sorting them on the basis of distance
    sorted_d = sorted(distances.items(), key=operator.itemgetter(1))
    #### End of STEP 3.2

    neighbors = []

    #### Start of STEP 3.3
    # Extracting top k neighbors
    for x in range(k):
        neighbors.append(sorted_d[x][0])
    #### End of STEP 3.3
    classVotes = {}

    #### Start of STEP 3.4
    # Calculating the most freq class in the neighbors
    for x in range(len(neighbors)):
        response = trainingSet.iloc[neighbors[x]][-1]

        if response in classVotes:
            classVotes[response] += 1
        else:
            classVotes[response] = 1
    #### End of STEP 3.4

    #### Start of STEP 3.5

```

```
sortedVotes = sorted(classVotes.items(), key=operator.itemgetter(1), reverse=True)
return(sortedVotes[0][0], neighbors)
#### End of STEP 3.5
```

HackathonScript1.py

```
def get_data_to_dictionary():
    data = open("ImputedNoErrors.csv", "rb")
    info = data.readlines() #
    data.close()

    categoryStr = info.pop(0)

    category = categoryStr.split(",")
    dataDict = {}
    catDict = {}

    tempCount = 0
    for cat in category:
        if cat == "MSICBD\r\n":
            dataDict.setdefault(str(cat[:-2]), [])
            catDict[tempCount] = str(cat[:-2])
        else:
            dataDict.setdefault(cat, [])
            catDict[tempCount] = cat
        tempCount += 1

    for j in range(len(info)):
        holder = info[j].split(",")
        counter = 0
        for col in dataDict:
            rCat = catDict[counter]
            hold = holder[counter]
            if hold == "":
                hold = "NA"
            if hold.find('\r\n') != -1:
                hold = hold[:-2]
            dataDict[rCat].append(hold)
            counter += 1
    return dataDict
```

dationary = get_data_to_dictionary()

readmit.py

import HackathonScript1

```

dationary = HackathonScript1.get_data_to_dictionary() #all info

clean_dationary = {}

#disputed_dationary = {"VisitLink": [], "LOS": [], "READMIT": []}

people = dationary["VisitLink"]
keys = dationary.keys()
ignoreList = []

for key in keys:
    clean_dationary.setdefault(key, [])

for i in range(len(people)):
    if dationary["READMIT"][i] == "1": #if readmit is yes
        ignoreList.append(people[i]) # put visit link on ignore list.
    if ignoreList.count(people[i]) == 0: #if the visitlinks does not appear on the ignore list
        if clean_dationary["VisitLink"].count(people[i]) == 0: #and the visitlink are not on
the output list
            for key in dationary:
                clean_dationary[key].append(dationary[key][i]) # add them to
output list

def createCSV(csv, dataDict):
    results = open(csv, "wb")
    for cat in dataDict:
        results.write(cat+",")
    results.write("\n")

    for i in range(len(dataDict["VisitLink"])):
        for cat in dataDict:
            results.write(dataDict[cat][i]+",")
        results.write("\n")

    results.close()

#createCSV("ImputedCleanData.csv", clean_dationary)

RemoveErrors.py

```

```

import HackathonScript1
import readmit

dationary = HackathonScript1.get_data_to_dictionary() #all info

clean_dationary = []

#disputed_dationary = {"VisitLink": [], "LOS": [], "READMIT": []}

people = dationary["VisitLink"]
keys = dationary.keys()
ignoreList = []

for key in keys:
    INE.setdefault(key, [])

for i in range(len(people)):
    if dationary["READMIT"][i] == "0": #if readmit is yes
        ignoreList.append(people[i]) # put visit link on ignore list.
    if ignoreList.count(people[i]) == 0: #if the visitlinks does not appear on the ignore list
        if clean_dationary.count(people[i]) == 0: #and the visitlink are not on the output
list
            clean_dationary.append(dationary["VisitLink"][i]) # add them to output
lis

for i in range(len(people)):
    if people not in clean_dationary:
        for key in INE:
            INE[key].append(dationary[key][i])

#for key in clean_dationary

#readmit.createCSV("ImputedNoErrors.csv", clean_dationary)

```

trendFinder.py

```

import HackathonScript1

```

```

import readmit

dationary = HackathonScript1.get_data_to_dictionary()

clean_dationary = {}
ones_zeros = {}

#disputed_dationary = {"VisitLink": [], "LOS": [], "READMIT": []}

people = dationary["VisitLink"]
keys = dationary.keys()
ignoreList = []

for key in keys:
    clean_dationary.setdefault(key, [])
    ones_zeros.setdefault(key, [])

for i in range(len(people)):
    if people.count(people[i]) > 1: #appear more than once
        # if dationary["READMIT"][i] == "0" and dationary["READMIT"][i+1] == "0":
        # if clean_dationary["VisitLink"].count(people[i]) == 0:
        #         for key in dationary:
        #             clean_dationary[key].append(dationary[key][i])
        if dationary["READMIT"][i] == "0" and dationary["READMIT"][i+1] == "1":
            #if ones_zeros["VisitLink"].count(people[i]) < 2:
            for key in dationary:
                ones_zeros[key].append(dationary[key][i])
                ones_zeros[key].append(dationary[key][i+1])

readmit.createCSV("UnderReadmit2Rows.csv", ones_zeros)
#readmit.createCSV("OverReadmit.csv", clean_dationary)

```

1281018_Predict_LOS

October 28, 2018

```
In [678]: # Importing libraries
import pandas as pd
import numpy as np
import os
import math
import operator

In [679]: import psycpg2 as pg
import pandas.io.sql as psql
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from statsmodels.formula.api import ols
from matplotlib import animation, rc
from sklearn.ensemble import RandomForestClassifier
from sklearn import preprocessing
import re
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
#Output plots in notebook
%matplotlib inline

from subprocess import check_output

#%matplotlib inline
from IPython.display import HTML
from IPython.display import set_matplotlib_formats
plt.rcParams['figure.autolayout'] = False
plt.rcParams['figure.figsize'] = 10, 6
plt.rcParams['axes.labelsize'] = 18
plt.rcParams['axes.titlesize'] = 20
plt.rcParams['font.size'] = 14
plt.rcParams['lines.linewidth'] = 2.0
plt.rcParams['lines.markersize'] = 8
plt.rcParams['legend.fontsize'] = 14

In [680]: pd.set_option('display.max_columns', 1000)
pd.set_option('display.width', 1000)
```

```

In [681]: os.chdir('A:\\Hackathon\\Data')

In [682]: df = pd.read_csv('1271018_CleanData.csv')
df.head(10)

Out [682]:
```

	Unnamed: 0	MEDINCSTQ	SUROPTOT	AGE	CCI	FTMDTF	CM_HTN_C	CM_ALCOHOL	PP086	M
0	0	4	17252	46	3	1	0	0	0	
1	1	2	9467	79	0	0	1	0	1	
2	2	4	24434	73	6	144	0	0	1	
3	3	4	53296	73	7	186	0	0	1	
4	4	3	60571	71	2	50	1	0	1	
5	5	4	8775	86	5	1	1	0	1	
6	6	3	60571	73	2	50	1	0	1	
7	7	4	8113	75	6	168	1	0	0	
8	8	3	8656	52	1	1	1	0	0	
9	9	3	24434	55	6	144	0	0	1	

```

In [683]: df.drop(['Unnamed: 0'], axis=1, inplace=True)

In [559]: df.groupby(['HOSPST']).size()

Out [559]: HOSPST
CA      4480
FL      3399
IA       223
MA       772
MD       433
NY      6112
WA      1377
WI       203
dtype: int64

In [202]: # df_ny = df.loc[((df['HOSPST']== 'NY'))]

In [305]: df_dum = pd.get_dummies(df)
# df_dum = df_dum.reset_index()

In [306]: # df_dum.dtypes

In [307]: df_dum.columns

Out [307]: Index(['MEDINCSTQ', 'SUROPTOT', 'AGE', 'CCI', 'FTMDTF', 'CM_HTN_C', 'CM_ALCOHOL', 'PP086', 'M', 'HOSPST_NY', 'HOSPST_WA', 'HOSPST_WI'],
dtype='object')

In [308]: # column_index(df, ['MEDINCSTQ', 'SUROPTOT', 'AGE', 'CCI', 'FTMDTF', 'CM_HTN_C', 'CM_ALCOHOL', 'PP086', 'M'])

In [309]: df_dum.columns[df_dum.isna().any()].tolist()

Out [309]: []

```

```

In [310]: from sklearn.tree import DecisionTreeClassifier
          from sklearn.model_selection import cross_val_score
          clf = DecisionTreeClassifier(random_state=0)

In [312]: x = df_dum.loc[:, ['MEDINCSTQ', 'SUROPTOT', 'AGE', 'CCI', 'FTMDTF', 'CM_HTN_C', 'CM_
          y = df_dum.loc[:, ['LOS']]
          # y = y.get_values()
          y = np.ravel(y)
          # x = pd.DataFrame(x)
          cla = clf.fit(x, y)

In [160]: for key in dict(zip(x.columns, clf.feature_importances_)):
          print(key+' ', '+str(dict(zip(x.columns, clf.feature_importances_))[k

MEDINCSTQ          0.00015141971099652183
SUROPTOT           0.0
AGE                0.0
CCI                6.489416185565222e-05
FTMDTF             0.0
CM_HTN_C           0.0
CM_ALCOHOL         0.0
PPO86              0.0
MAPP12n            0.0
BDTOT              0.0
TOTCHG_X           0.0
CHC                0.0
READMIT            0.0
TETOT              0.00012978832371130444
HMO86              0.0
SUROPIP            0.0
YEAR               0.00023794526013739146
MAPP3n             0.0
CM_LYTES           6.489416185565222e-05
CM_ARTH            0.0
Sepsis             0.0
MAPP13n            6.489416185565222e-05
NPR                0.0001260800858909814
CM_VALVE           0.0
MAPP6n             0.0
FTRES              0.00022064015030921755
AWEEKEND           0.0
CM_ANEMDEF         6.489416185565222e-05
MSICBD             0.0
CM_COAG            0.0
CM_LYMPH           0.0
PAY1               0.0
PE                 0.0
MAPP9n             0.0

```

LIVRHOS	0.0	
NCHRONIC	0.0004326277457043481	
CM_METS	0.0	
CM_WGHTLOSS	0.00010383065896904352	
MAPP1n	0.0	
GENHOS	0.0	
CM_PERIVASC	0.0	
MAPP8n	0.0	
MI	0.0	
CM_LIVER	0.0	
CM_PARA	0.0	
CM_OBESE	0.0	
MAPP16n	0.0	
CM_PULMCIRC	0.0	
DVT	0.0	
MSICHOS	0.0	
DISPUNIFORM	0.0	
SUROPOP	0.0	
Infection	0.0	
FTRNTF	0.0	
ATYPE	0.0	
NDX	0.0	
ADC	0.0	
MAPP7n	9.875847160561326e-05	
CM_DMCX	0.0	
VEM	6.489416185565222e-05	
CM_HYPOTHY	0.0	
ADMTOT	0.0002753085654482215	
LOS	0.9974782452253423	
MAPP2n	6.489416185565222e-05	
VisitLink	7.478279604318015e-05	
CM_CHRNLUNG	0.0	
CM_PSYCH	0.0	
CM_ULCER	0.0	
MAPP5n	6.489416185565222e-05	
CM_DRUG	0.0	
CM_RENLFAIL	0.0	
MAPP10n	0.0	
RACE	0.0	
Surgery_Complications	0.0	
Cardiac	0.0	
CM_DM	0.0	
CM_NEURO	0.0	
UTI	0.0	
CM_DEPRESS	0.0	
CM_BLDLOSS	0.0	
FEMALE	0.0	
CM_AIDS	0.0	

CM_TUMOR	0.0
MAPP11n	0.0
CM_CHF	0.0
PNA	0.0
HOSPST_CA	0.0
HOSPST_FL	0.0
HOSPST_IA	0.0
HOSPST_MA	0.0
HOSPST_MD	8.65255491408696e-05
HOSPST_NY	0.00012978832371130444
HOSPST_WA	0.0
HOSPST_WI	0.0

```
In [161]: sorted_x = sorted(dict(zip(x.columns, clf.feature_importances_)).items(), key=lambda
        holder = sorted_x
        for key in holder:
            print(key)
```

```
('SUROPTOT', 0.0)
('AGE', 0.0)
('FTMDTF', 0.0)
('CM_HTN_C', 0.0)
('CM_ALCOHOL', 0.0)
('PPO86', 0.0)
('MAPP12n', 0.0)
('BDTOT', 0.0)
('TOTCHG_X', 0.0)
('CHC', 0.0)
('READMIT', 0.0)
('HMO86', 0.0)
('SUROPIP', 0.0)
('MAPP3n', 0.0)
('CM_ARTH', 0.0)
('Sepsis', 0.0)
('CM_VALVE', 0.0)
('MAPP6n', 0.0)
('A WEEKEND', 0.0)
('MSICBD', 0.0)
('CM_COAG', 0.0)
('CM_LYMPH', 0.0)
('PAY1', 0.0)
('PE', 0.0)
('MAPP9n', 0.0)
('LIVRHOS', 0.0)
('CM_METS', 0.0)
('MAPP1n', 0.0)
('GENHOS', 0.0)
```

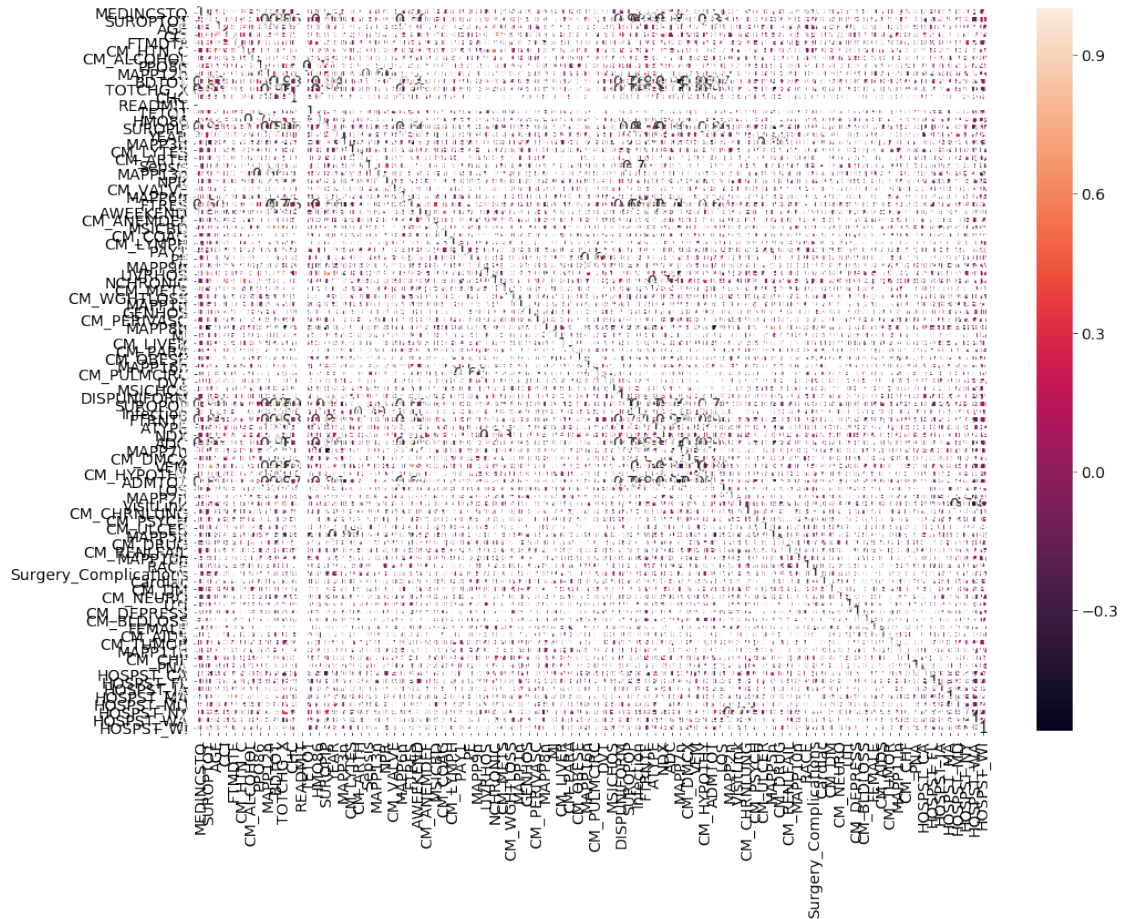
('CM_PERIVASC', 0.0)
('MAPP8n', 0.0)
('MI', 0.0)
('CM_LIVER', 0.0)
('CM_PARA', 0.0)
('CM_OBESE', 0.0)
('MAPP16n', 0.0)
('CM_PULMCIRC', 0.0)
('DVT', 0.0)
('MSICHOS', 0.0)
('DISPUNIFORM', 0.0)
('SUROPOP', 0.0)
('Infection', 0.0)
('FTRNTF', 0.0)
('ATYPE', 0.0)
('NDX', 0.0)
('ADC', 0.0)
('CM_DMCX', 0.0)
('CM_HYPOTHY', 0.0)
('CM_CHRNLUNG', 0.0)
('CM_PSYCH', 0.0)
('CM_ULCER', 0.0)
('CM_DRUG', 0.0)
('CM_RENLFAIL', 0.0)
('MAPP10n', 0.0)
('RACE', 0.0)
('Surgery_Complications', 0.0)
('Cardiac', 0.0)
('CM_DM', 0.0)
('CM_NEURO', 0.0)
('UTI', 0.0)
('CM_DEPRESS', 0.0)
('CM_BLDLOSS', 0.0)
('FEMALE', 0.0)
('CM_AIDS', 0.0)
('CM_TUMOR', 0.0)
('MAPP11n', 0.0)
('CM_CHF', 0.0)
('PNA', 0.0)
('HOSPST_CA', 0.0)
('HOSPST_FL', 0.0)
('HOSPST_IA', 0.0)
('HOSPST_MA', 0.0)
('HOSPST_WA', 0.0)
('HOSPST_WI', 0.0)
('CCI', 6.489416185565222e-05)
('CM_LYTES', 6.489416185565222e-05)
('MAPP13n', 6.489416185565222e-05)

```
('CM_ANEMDEF', 6.489416185565222e-05)
('VEM', 6.489416185565222e-05)
('MAPP2n', 6.489416185565222e-05)
('MAPP5n', 6.489416185565222e-05)
('VisitLink', 7.478279604318015e-05)
('HOSPST_MD', 8.65255491408696e-05)
('MAPP7n', 9.875847160561326e-05)
('CM_WGHTLOSS', 0.00010383065896904352)
('NPR', 0.0001260800858909814)
('TETOT', 0.00012978832371130444)
('HOSPST_NY', 0.00012978832371130444)
('MEDINCSTQ', 0.00015141971099652183)
('FTRES', 0.00022064015030921755)
('YEAR', 0.00023794526013739146)
('ADMTOT', 0.0002753085654482215)
('NCHRONIC', 0.0004326277457043481)
('LOS', 0.9974782452253423)
```

```
In [208]: #Correlation Matrix
```

```
corr = x.corr()
corr = (corr)
plt.figure(figsize=(16, 12))
sns.heatmap(corr,
             xticklabels=corr.columns.values,
             yticklabels=corr.columns.values,
             annot=True, linewidths=1.5)
```

```
Out[208]: <matplotlib.axes._subplots.AxesSubplot at 0x2beff0cfc88>
```



```
In [209]: corr_left = pd.DataFrame(corr['LOS'].drop('LOS'))
          corr_left.sort_values(by = 'LOS', ascending = False)
```

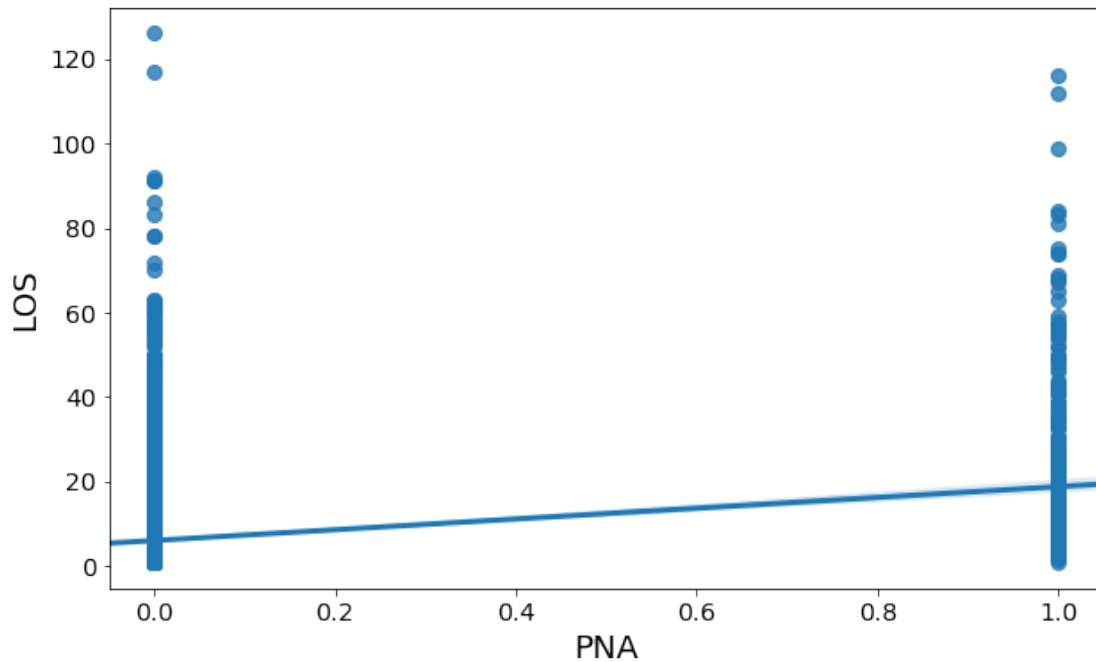
Out[209]:	LOS
NPR	0.551744
NDX	0.415752
Sepsis	0.413473
DISPUNIFORM	0.295833
PNA	0.295265
Infection	0.260194
CM_WGHTLOSS	0.232168
CM_LYTES	0.228160
NCHRONIC	0.135252
CM_COAG	0.133619
DVT	0.131560
CM_ANEMDEF	0.115522
PE	0.101801
CM_PULMCIRC	0.095525
MI	0.070739

AWEEKEND	0.070599
CM_CHF	0.064290
Surgery_Complications	0.063087
CM_METS	0.060781
CM_RENLFAIL	0.056137
CM_PARA	0.054208
CM_PSYCH	0.052740
Cardiac	0.050246
CCI	0.049661
CM_NEURO	0.047083
CM_ALCOHOL	0.044899
CM_BLDLOSS	0.043135
CM_DRUG	0.039098
UTI	0.036643
MAPP7n	0.034579
...	...
ADMTOT	0.000028
CM_AIDS	-0.001055
MAPP3n	-0.001089
CHC	-0.001504
HOSPST_IA	-0.001979
CM_TUMOR	-0.003354
MAPP11n	-0.004913
TETOT	-0.004923
PPO86	-0.005652
CM_ARTH	-0.006293
MAPP9n	-0.009549
CM_HYPOTHY	-0.010566
LIVRHOS	-0.010740
MAPP12n	-0.010930
HOSPST_MA	-0.011134
YEAR	-0.011291
AGE	-0.011672
SUROPIP	-0.013026
MAPP8n	-0.014767
MSICHOS	-0.017992
CM_LIVER	-0.018901
MAPP2n	-0.019121
SUROPTOT	-0.020473
PAY1	-0.022183
SUROPOP	-0.023029
MEDINCSTQ	-0.033718
HOSPST_WA	-0.038437
FEMALE	-0.067682
ATYPE	-0.093427
READMIT	NaN

[93 rows x 1 columns]

```
In [382]: sns.regplot(x=x_npr['PNA'], y=x_npr['LOS'])
```

```
Out[382]: <matplotlib.axes._subplots.AxesSubplot at 0x2be8c67ceb8>
```

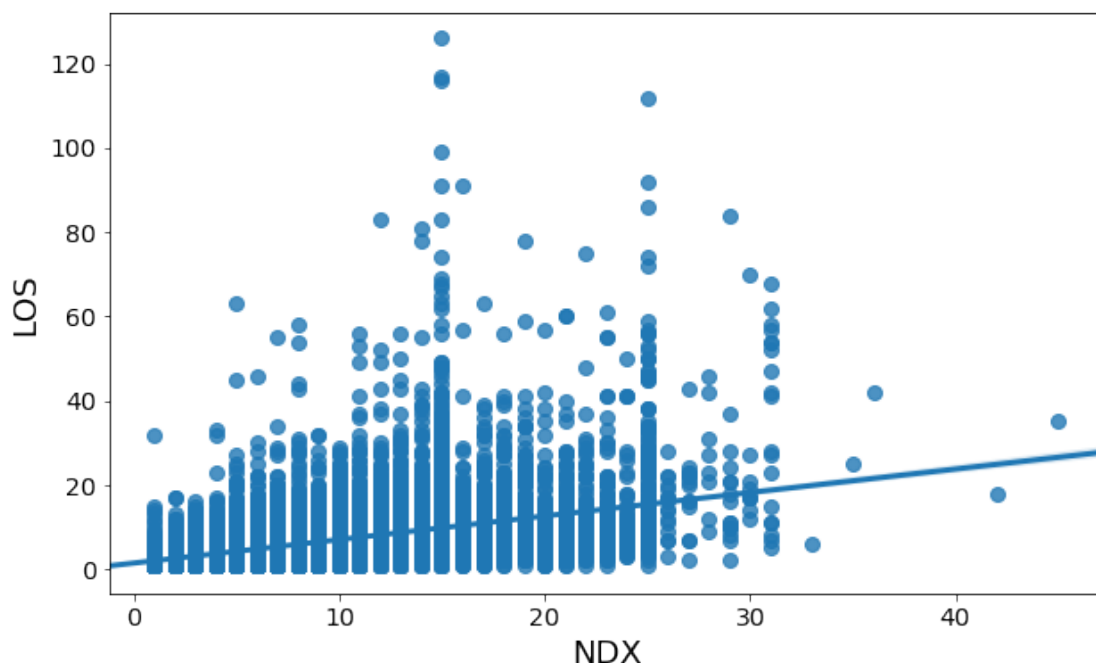


```
In [383]: x_npr = x.loc[((df['LOS'] <= 140) & (df['NPR'] <= 25))]
```

```
In [376]: x_npr = x_npr.loc[((x_npr['NDX'] <= 35))]
```

```
In [384]: sns.regplot(x=x_npr['NDX'], y=x_npr['LOS'])
```

```
Out[384]: <matplotlib.axes._subplots.AxesSubplot at 0x2be8c96c400>
```



```
In [391]: formulae = "LOS~NPR+NDX+Sepsis+DISPUNIFORM+PNA+CM_WGHTLOSS+CM_LYTES+NCHRONIC+CM_COAG"
```

```
In [392]: # Regression
```

```
model = smf.ols(formula = formulae, data = x_npr).fit()
model.summary()
```

```
Out [392]: <class 'statsmodels.iolib.summary.Summary'>
"""
```

OLS Regression Results

```
=====
Dep. Variable:          LOS    R-squared:                0.477
Model:                  OLS    Adj. R-squared:         0.477
Method:                 Least Squares    F-statistic:          1291.
Date:                   Sun, 28 Oct 2018    Prob (F-statistic):    0.00
Time:                   05:45:21    Log-Likelihood:       -50553.
No. Observations:       16994    AIC:                  1.011e+05
Df Residuals:           16981    BIC:                  1.012e+05
Df Model:                12
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.9823	0.084	11.682	0.000	0.817	1.147
NPR	0.7600	0.014	53.384	0.000	0.732	0.788
NDX	0.2443	0.014	17.553	0.000	0.217	0.272

Sepsis	7.4505	0.222	33.490	0.000	7.014	7.887
DISPUNIFORM	0.4346	0.021	21.169	0.000	0.394	0.475
PNA	5.2517	0.245	21.412	0.000	4.771	5.732
CM_WGHTLOSS	2.8540	0.196	14.594	0.000	2.471	3.237
CM_LYTES	0.9410	0.107	8.817	0.000	0.732	1.150
NCHRONIC	-0.2866	0.022	-12.807	0.000	-0.331	-0.243
CM_COAG	0.3483	0.152	2.299	0.022	0.051	0.645
DVT	3.7580	0.542	6.932	0.000	2.695	4.821
CM_ANEMDEF	0.5678	0.115	4.950	0.000	0.343	0.793
PE	2.2665	0.475	4.776	0.000	1.336	3.197

Omnibus:	17855.518	Durbin-Watson:	1.935
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2814943.856
Skew:	4.974	Prob(JB):	0.00
Kurtosis:	65.261	Cond. No.	190.

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified

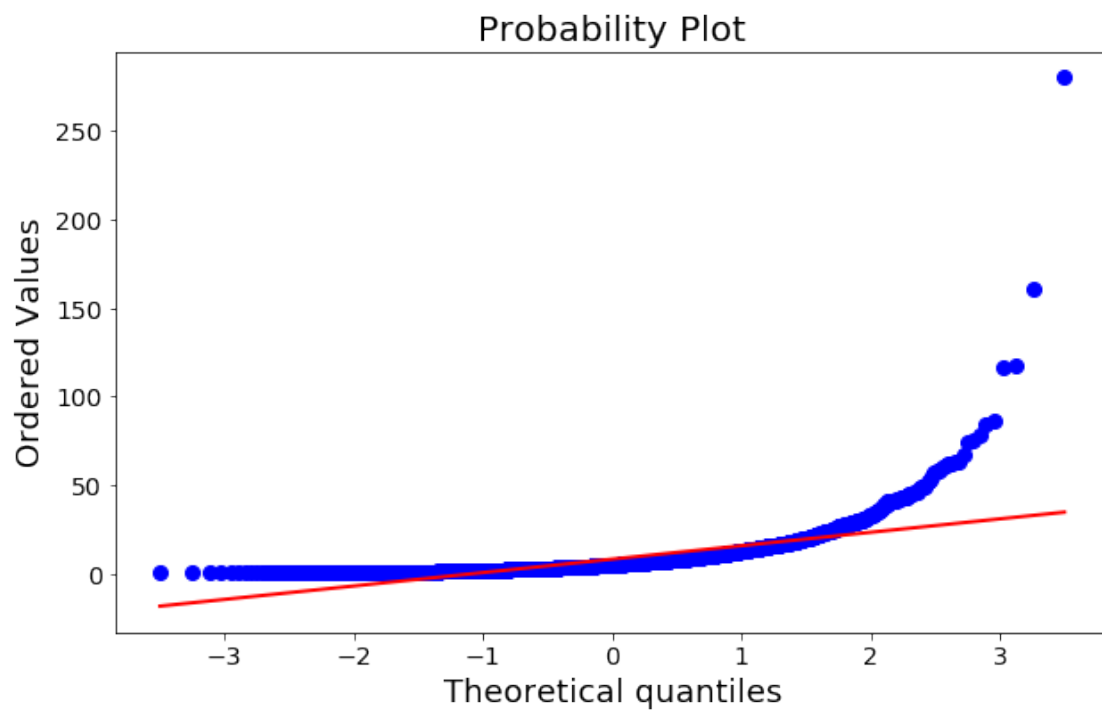
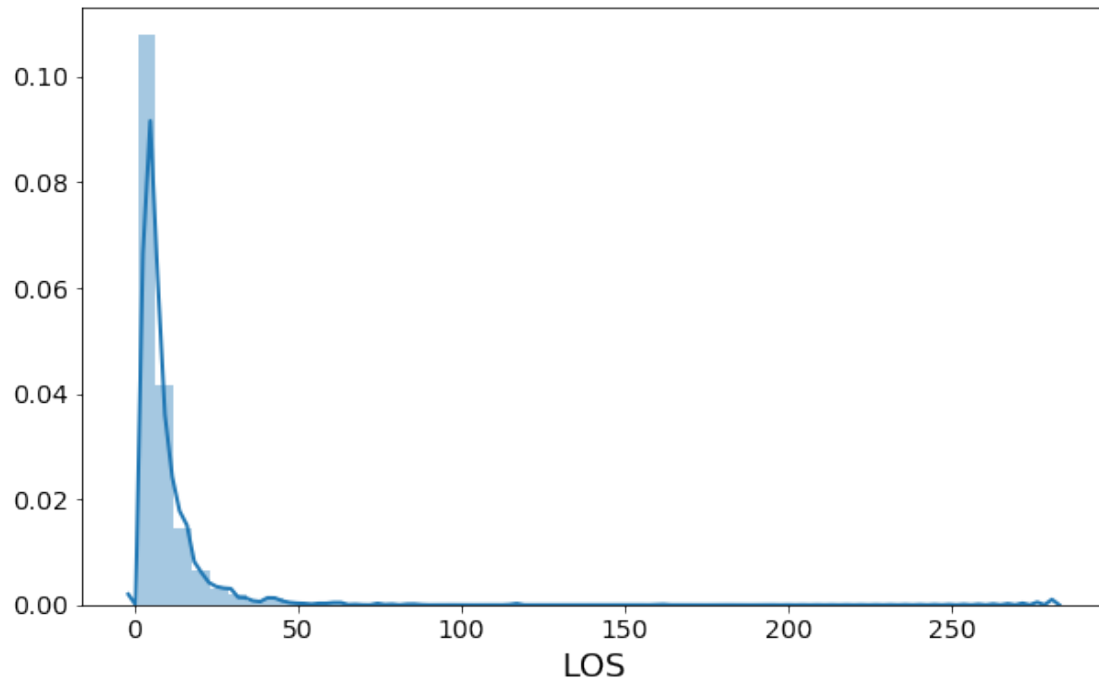
```
In [636]: df.groupby(['ATYPE']).size().sort_values(ascending=False)
```

```
Out[636]: ATYPE
3      13409
1       2926
2        664
dtype: int64
```

```
In [684]: df = df.loc[((df['ATYPE'] == 1))]
```

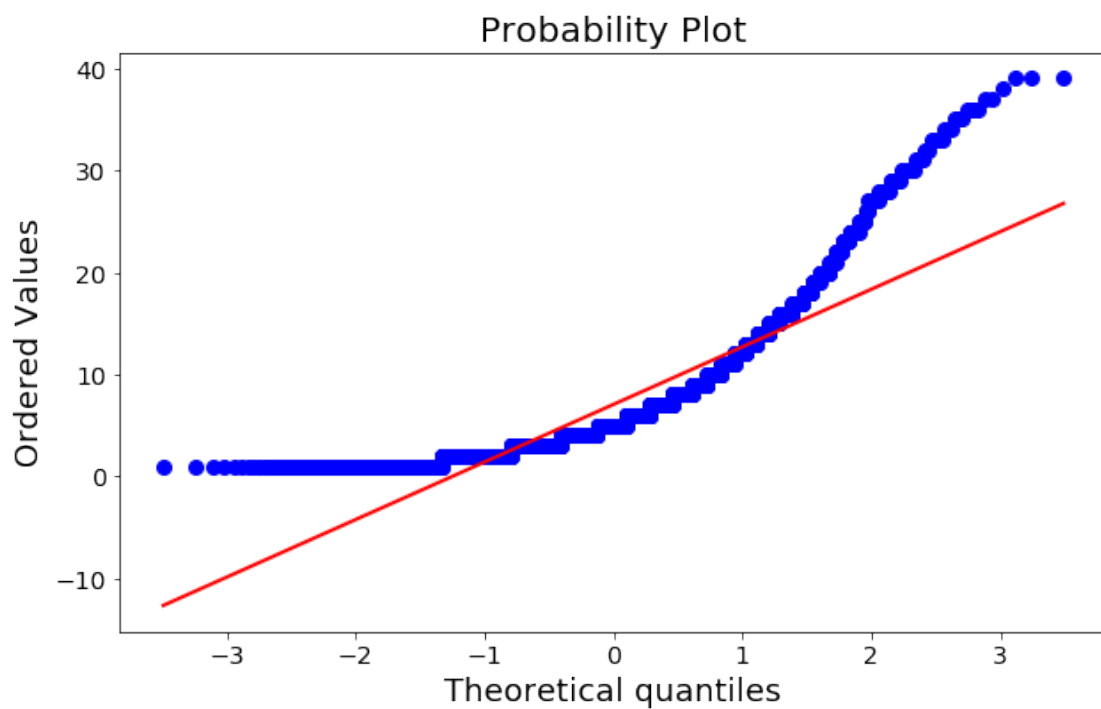
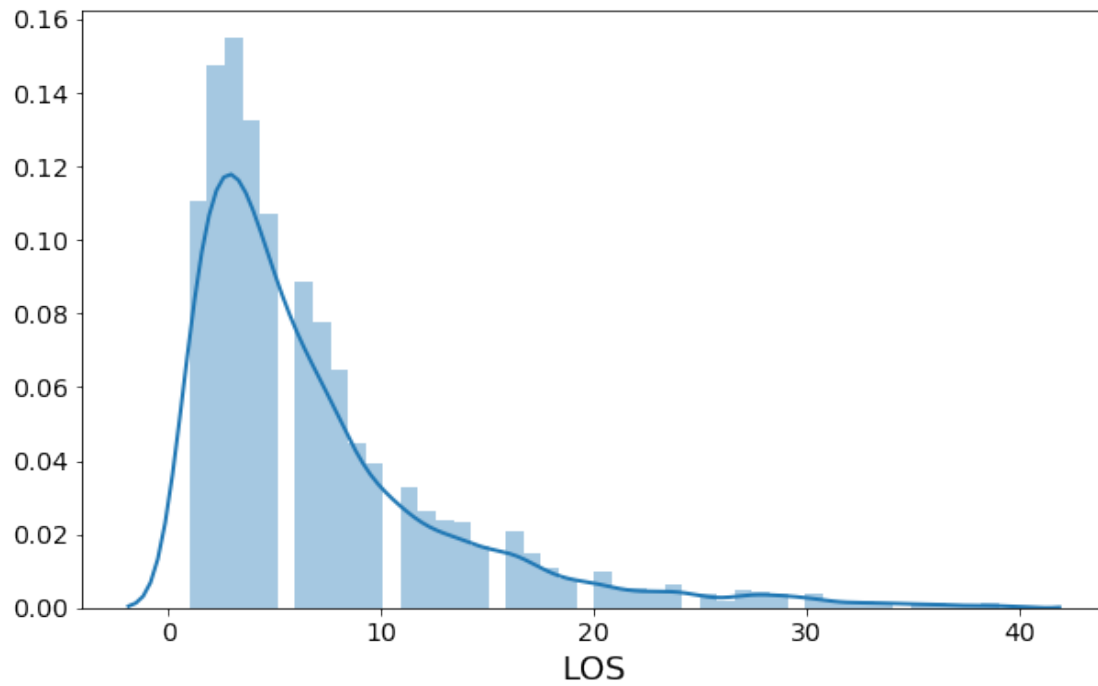
```
In [690]: df.drop(['ATYPE'], axis=1, inplace=True)
```

```
In [685]: # #histogram and t normal probability plot
sns.distplot(df['LOS']);
fig = plt.figure()
res = stats.probplot(df['LOS'], plot=plt)
```



```
In [686]: df =df.loc[((df['LOS'] <= 40))]
```

```
In [687]: # #histogram and_t normal probability plot
sns.distplot(df['LOS']);
fig = plt.figure()
res = stats.probplot(df['LOS'], plot=plt)
```



```
In [688]: df = df.loc[:, ['MEDINCSTQ', 'SUROPTOT', 'AGE', 'CCI', 'FTMDTF', 'CM_HTN_C', 'CM_ALCOHOL']]
```

```
In [689]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 2877 entries, 1 to 16998
Data columns (total 84 columns):
MEDINCSTQ                2877 non-null int64
SUROPTOT                 2877 non-null int64
AGE                      2877 non-null int64
CCI                      2877 non-null int64
FTMDTF                   2877 non-null int64
CM_HTN_C                 2877 non-null int64
CM_ALCOHOL               2877 non-null int64
PP086                    2877 non-null int64
MAPP12n                  2877 non-null int64
BDTOT                    2877 non-null int64
CHC                      2877 non-null int64
TETOT                    2877 non-null int64
HMO86                    2877 non-null int64
SUROPIP                  2877 non-null int64
YEAR                     2877 non-null int64
MAPP3n                   2877 non-null int64
CM_LYTES                 2877 non-null int64
CM_ARTH                  2877 non-null int64
Sepsis                   2877 non-null int64
MAPP13n                  2877 non-null int64
NPR                      2877 non-null int64
CM_VALVE                 2877 non-null int64
MAPP6n                   2877 non-null int64
FTRES                    2877 non-null int64
AWEEKEND                 2877 non-null int64
CM_ANEMDEF               2877 non-null int64
MSICBD                   2877 non-null int64
CM_COAG                  2877 non-null int64
CM_LYMPH                 2877 non-null int64
PAY1                     2877 non-null int64
PE                       2877 non-null int64
MAPP9n                   2877 non-null int64
LIVRHOS                  2877 non-null int64
NCHRONIC                 2877 non-null int64
CM_METS                  2877 non-null int64
CM_WGHTLOSS              2877 non-null int64
MAPP1n                   2877 non-null int64
GENHOS                   2877 non-null int64
```

CM_PERIVASC	2877 non-null int64
MAPP8n	2877 non-null int64
MI	2877 non-null int64
CM_LIVER	2877 non-null int64
CM_PARA	2877 non-null int64
CM_OBESE	2877 non-null int64
MAPP16n	2877 non-null int64
CM_PULMCIRC	2877 non-null int64
DVT	2877 non-null int64
MSICHOS	2877 non-null int64
DISPUNIFORM	2877 non-null int64
SUROPOP	2877 non-null int64
Infection	2877 non-null int64
FTRNTF	2877 non-null int64
ATYPE	2877 non-null int64
NDX	2877 non-null int64
ADC	2877 non-null int64
MAPP7n	2877 non-null int64
CM_DMCX	2877 non-null int64
VEM	2877 non-null int64
CM_HYPOTHY	2877 non-null int64
ADMTOT	2877 non-null int64
LOS	2877 non-null int64
MAPP2n	2877 non-null int64
CM_CHRNLUNG	2877 non-null int64
CM_PSYCH	2877 non-null int64
CM_ULCER	2877 non-null int64
MAPP5n	2877 non-null int64
CM_DRUG	2877 non-null int64
CM_RENLFAIL	2877 non-null int64
MAPP10n	2877 non-null int64
RACE	2877 non-null int64
Surgery_Complications	2877 non-null int64
Cardiac	2877 non-null int64
CM_DM	2877 non-null int64
CM_NEURO	2877 non-null int64
UTI	2877 non-null int64
CM_DEPRESS	2877 non-null int64
CM_BLDLOSS	2877 non-null int64
FEMALE	2877 non-null int64
CM_AIDS	2877 non-null int64
CM_TUMOR	2877 non-null int64
MAPP11n	2877 non-null int64
CM_CHF	2877 non-null int64
PNA	2877 non-null int64
HOSPST	2877 non-null object

dtypes: int64(83), object(1)

memory usage: 1.9+ MB

```

In [644]: # Ignore warnings :# Ignore
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import time
from sklearn.manifold import TSNE
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import AffinityPropagation, KMeans
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.datasets.samples_generator import make_blobs
from itertools import cycle
from sklearn import preprocessing
import seaborn as sns
# from ggplot import *
import seaborn as sns
import pickle
# import missingno as msno
import scipy
# from tqdm import tqdm
from sklearn.decomposition import PCA
# import kmodes
from sklearn.preprocessing import MinMaxScaler , StandardScaler, Imputer
le = preprocessing.LabelEncoder()

from sklearn.linear_model import LinearRegression,Ridge,Lasso,RidgeCV, ElasticNet
from sklearn.ensemble import RandomForestRegressor,BaggingRegressor,GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.neighbors import KNeighborsRegressor

from sklearn.model_selection import GridSearchCV , RandomizedSearchCV, KFold , cross_val_score
from sklearn.metrics import mean_squared_log_error,mean_squared_error, r2_score,mean_absolute_error

In [645]: def get_unique_values(data, feature):
            return np.unique(data[feature])

def Encoding(key_order):
    val = np.arange(0, len(key_order), 1)
    d = {}
    for i, key in enumerate(key_order):

```

```

        d[key] = val[i]
    return d

def Transform(encoder, series):
    return series.map(encoder)

def Decoding(encoder, series):
    decoder = {v:k for k,v in encoder.items()}
    return series.map(decoder)

def getNumericAndCategoricalFeatures(data):
    numeric_features = data.describe().columns.tolist()
    categorical_features = data.columns.tolist()
    for col in numeric_features:
        categorical_features.remove(col)
    return numeric_features, categorical_features

In [646]: categorical_features_keys_dict = {}
categorical_features_keys_dict['HOSPST'] = Encoding((get_unique_values(df, 'HOSPST'))

In [647]: def splitData(featureToPredict, data, categorical_features_keys_dict):
    data_y = df[featureToPredict]
    df.drop([featureToPredict], axis=1, inplace=True)

    cols = df.columns.tolist()
    data_X = df.copy()
    numeric_features, categorical_features = getNumericAndCategoricalFeatures(df)
    for cf in categorical_features:
        data_X[cf] = Transform(categorical_features_keys_dict[cf], data_X[cf])
    #     sc_X = StandardScaler()
    #     data_X = sc_X.fit_transform(data_X)
    #     sc_y = StandardScaler()
    #     data_y = sc_y.fit_transform(np.array(data_y, dtype=float).reshape(-1,1))
    data_X = pd.DataFrame(data_X, columns=cols)
    data_y = pd.DataFrame(data_y, columns=[featureToPredict])
    return data_X, data_y, sc_X, sc_y

In [648]: # df.columns[df.isna().any()].tolist()
# df.info()

In [649]: data_X, data_y, sc_X, sc_y = splitData('LOS', df.copy(), categorical_features_keys_d

In [650]: X = data_X.copy()
y = data_y.copy()

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)

In [651]: def evaluate_predicted_values(y_test, y_pred):
    mse = mean_squared_error(y_test, y_pred)

```

```

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred)**0.5
r2 = r2_score(y_test, y_pred)

```

```

return mse, mae, rmse, r2

```

```

def print_eval_metrics(mse, mae, rmse, r2):
    print("Mean Squared Error (MSE) is =", mse)
    print("Mean Absolute Error (MAE) is =", mae)
    print("Root Mean Squared Error (RMSE) is =", rmse)
    print("R2 score is =", r2)

```

In [652]: *# Random search has shown to perform better than grid search in most of the cases*
(Ex: <https://www.kaggle.com/dkim1992/grid-search-vs-random-search>)

```

def bestModelSelectedUsingRandomSearch(model, param_dist, n_iter_search, cv, X_train, y_train, X_test, y_test):
    random_search = RandomizedSearchCV(model, param_distributions=param_dist, n_iter=n_iter_search, cv=cv)
    random_search.fit(X_train, y_train)
    y_pred = random_search.predict(X_test)
    mse, mae, rmse, r2 = evaluate_predicted_values(y_test, y_pred)
    return random_search.best_estimator_, random_search.best_params_, random_search.best_score_

```

In [653]: model_score_dict = {}

```

In [654]: lr_model = LinearRegression()
param_dist = {}
n_iter_search = 1
cv = 5
best_ols_model, best_ols_param, best_ols_score, ols_test_mse, ols_test_mae, ols_test_r2 = bestModelSelectedUsingRandomSearch(lr_model, param_dist, n_iter_search, cv, X_train, y_train, X_test, y_test)
print("***** Ordinary Linear Regression *****")
print("Best set of parameters is :", best_ols_param)
print("Best score is :", best_ols_score)
print("Test MSE is :", ols_test_mse)
print("Test MAE is :", ols_test_mae)
print("Test R2 is :", ols_test_r2)
print("*****")
model_score_dict['Linear Regression'] = [best_ols_score, ols_test_mse, ols_test_mae, ols_test_r2]

```

Fitting 5 folds for each of 1 candidates, totalling 5 fits

```

***** Ordinary Linear Regression *****
Best set of parameters is : {}
Best score is : 0.471215675790095
Test MSE is : 20.808634032068387
Test MAE is : 3.203949201862658
Test R2 is : 0.5666082748625009
*****

```

[Parallel(n_jobs=1)]: Done 5 out of 5 | elapsed: 0.0s finished

```
In [655]: lasso_model = Lasso(normalize=True)
alpha_space = np.exp(np.arange(0.001,-10,-0.5))
param_dist = {'alpha': alpha_space}
n_iter_search = 20
cv = 5
best_lasso_model, best_lasso_param, best_lasso_score, lasso_test_mse, lasso_test_mae
print("***** Lasso Regression *****")
print("Best set of parameters is :", best_lasso_param)
print("Best score is :", best_lasso_score)
print("Test MSE is :", lasso_test_mse)
print("Test MAE is :", lasso_test_mae)
print("Test R2 is :", lasso_test_r2)
print("*****")
model_score_dict['Lasso Regression'] = [best_lasso_score, lasso_test_mse, lasso_test_mae]
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

***** Lasso Regression *****

Best set of parameters is : {'alpha': 0.0015049433841407829}

Best score is : 0.48261810556863943

Test MSE is : 20.60805218286239

Test MAE is : 3.1780635856864343

Test R2 is : 0.5707858923613087

[Parallel(n_jobs=1)]: Done 100 out of 100 | elapsed: 2.1s finished

```
In [656]: adaboost_model = AdaBoostRegressor()
n_estimators = np.arange(5, 125, 5)
param_dist = {'n_estimators': n_estimators}
n_iter_search = 20
cv = 5
best_adaboost_model, best_adaboost_param, best_adaboost_score, adaboost_test_mse, adaboost_test_mae
print("***** Adaboost Regression *****")
print("Best set of parameters is :", best_adaboost_param)
print("Best score is :", best_adaboost_score)
print("Test MSE is :", adaboost_test_mse)
print("Test MAE is :", adaboost_test_mae)
print("Test R2 is :", adaboost_test_r2)
print("*****")
model_score_dict['Adaboost Regression'] = [best_adaboost_score, adaboost_test_mse, adaboost_test_mae]
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

***** Adaboost Regression *****

Best set of parameters is : {'n_estimators': 10}

Best score is : 0.36302750997964844

Test MSE is : 26.332625507404234

Test MAE is : 3.7611755190869514

Test R2 is : 0.45155736900048493

[Parallel(n_jobs=1)]: Done 100 out of 100 | elapsed: 31.3s finished

```
In [657]: gbr_model = GradientBoostingRegressor()
          n_estimators = np.arange(5, 150, 5)
          param_dist = {'n_estimators': n_estimators}
          n_iter_search = 20
          cv = 5
          best_gbr_model, best_gbr_param, best_gbr_score, gbr_test_mse, gbr_test_mae, gbr_test_r2 = cross_validate(gbr_model, X, y, cv=cv, param_grid=param_dist, n_iter_search=n_iter_search)
          print("***** Gradient Boosting Regression *****")
          print("Best set of parameters is :", best_gbr_param)
          print("Best score is :", best_gbr_score)
          print("Test MSE is :", gbr_test_mse)
          print("Test MAE is :", gbr_test_mae)
          print("Test R2 is :", gbr_test_r2)
          print("*****")
          model_score_dict['Gradient Boosting Regression'] = [best_gbr_score, gbr_test_mse, gbr_test_mae, gbr_test_r2]
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

[Parallel(n_jobs=1)]: Done 100 out of 100 | elapsed: 25.5s finished

```
***** Gradient Boosting Regression *****
Best set of parameters is : {'n_estimators': 120}
Best score is : 0.48620628201769767
Test MSE is : 20.704969269696953
Test MAE is : 3.1295858495009714
Test R2 is : 0.5687673522017864
*****
```

```
In [658]: rf_model = RandomForestRegressor()
          n_estimators = np.arange(5, 150, 5)
          param_dist = {'n_estimators': n_estimators}
          n_iter_search = 5
          cv = 5
          best_rf_estimator, best_rf_param, best_rf_score, rf_test_mse, rf_test_mae, rf_test_r2 = cross_validate(rf_model, X, y, cv=cv, param_grid=param_dist, n_iter_search=n_iter_search)
          print("***** Random Forest Regression *****")
          print("Best set of parameters is :", best_rf_param)
          print("Best score is :", best_rf_score)
          print("Test MSE is :", rf_test_mse)
          print("Test MAE is :", rf_test_mae)
          print("Test R2 is :", rf_test_r2)
```

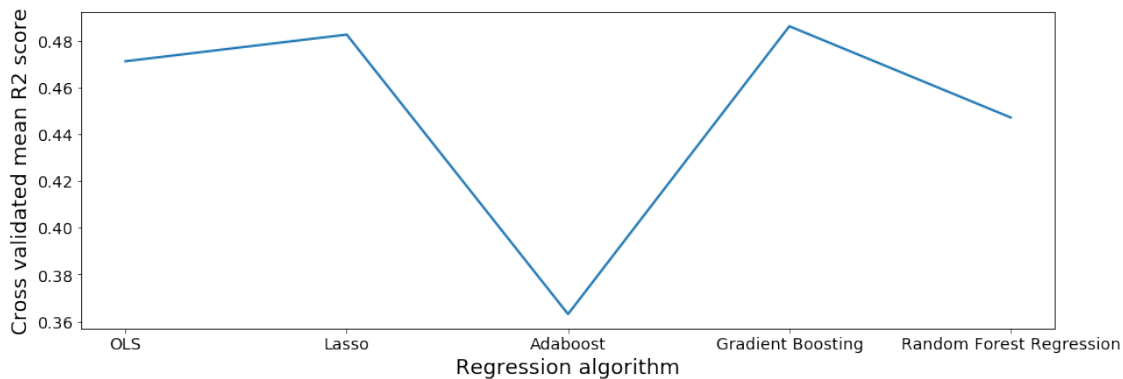
```
print("*****")
model_score_dict['Random Forest Regression'] = [best_rf_score, rf_test_mse, rf_test_r
```

Fitting 5 folds for each of 5 candidates, totalling 25 fits

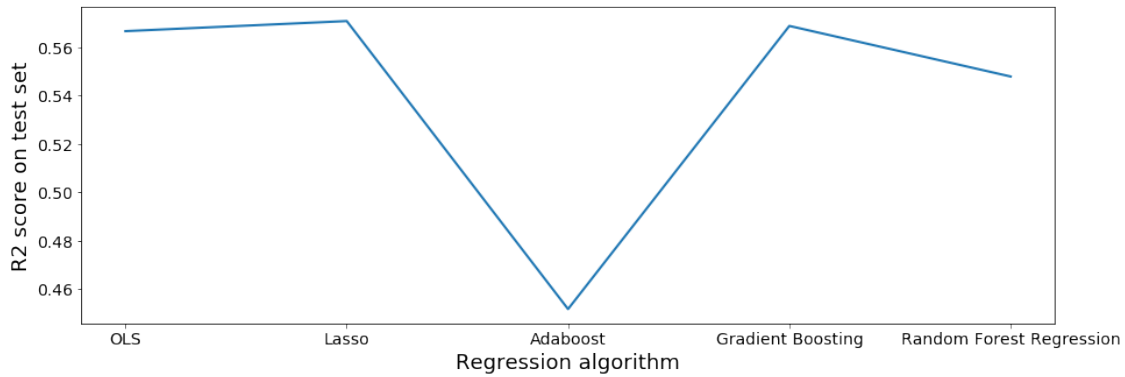
[Parallel(n_jobs=1)]: Done 25 out of 25 | elapsed: 27.6s finished

```
***** Random Forest Regression *****
Best set of parameters is : {'n_estimators': 95}
Best score is : 0.44715639724844825
Test MSE is : 21.71019294398276
Test MAE is : 3.221546052631579
Test R2 is : 0.54783106096439
*****
```

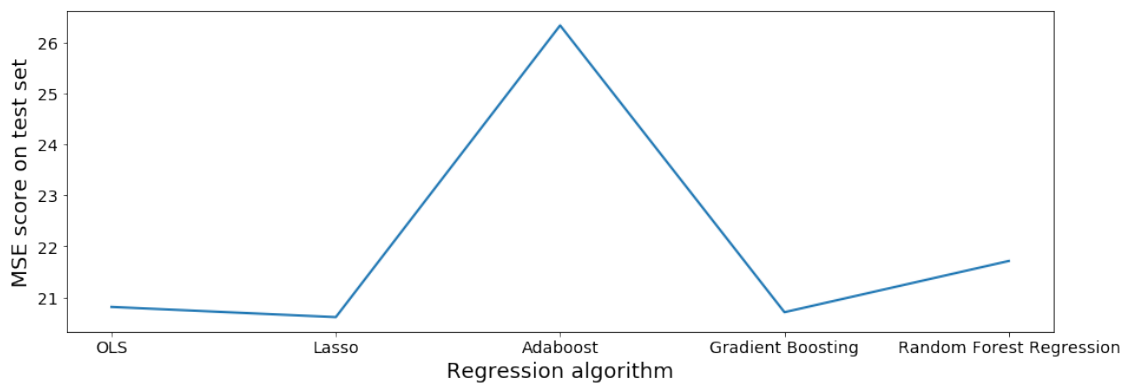
```
In [659]: models = ['OLS', 'Lasso', 'Adaboost', 'Gradient Boosting', 'Random Forest Regression']
models_idx = np.arange(1, len(models)+1, 1)
r2_score = [best_ols_score, best_lasso_score, best_adaboost_score, best_gbr_score, best_rf_score]
plt.figure(figsize=(15,5))
plt.plot(models_idx, r2_score)
plt.xticks(models_idx, models)
plt.xlabel('Regression algorithm')
plt.ylabel('Cross validated mean R2 score')
plt.show()
```



```
In [660]: r2_test_score = [ols_test_r2, lasso_test_r2, adaboost_test_r2, gbr_test_r2, rf_test_r2]
plt.figure(figsize=(15,5))
plt.plot(models_idx, r2_test_score)
plt.xticks(models_idx, models)
plt.xlabel('Regression algorithm')
plt.ylabel('R2 score on test set')
plt.show()
```



```
In [661]: mse_test_score = [ols_test_mse, lasso_test_mse, adaboost_test_mse, gbr_test_mse, rf_test_mse]
plt.figure(figsize=(15,5))
plt.plot(models_idx, mse_test_score)
plt.xticks(models_idx, models)
plt.xlabel('Regression algorithm')
plt.ylabel('MSE score on test set')
plt.show()
```



```
In [662]: pd.DataFrame(model_score_dict, index=['CV R2 score', 'Test MSE', 'Test MAE', 'Test R2'])
```

```
Out[662]:
```

	CV R2 score	Test MSE	Test MAE	Test R2
Linear Regression	0.471216	20.808634	3.203949	0.566608
Lasso Regression	0.482618	20.608052	3.178064	0.570786
Adaboost Regression	0.363028	26.332626	3.761176	0.451557
Gradient Boosting Regression	0.486206	20.704969	3.129586	0.568767
Random Forest Regression	0.447156	21.710193	3.221546	0.547831

```
In [663]: gbr_model.fit(X_train, y_train)
y_pred=gbr_model.predict(X_test)
```

```
In [668]: X_test['NEW_LOS'] = y_pred
```

```
In [676]: X_test.head()
```

```
Out[676]:
```

	MEDINCSTQ	SUROPTOT	AGE	CCI	FTMDTF	CM_HTN_C	CM_ALCOHOL	PP086	MAPP12n	BL
716	3	2859	55	0	2	0	0	1	2	
7159	1	19691	67	2	650	0	0	1	2	
2687	2	10665	82	2	0	1	0	0	2	
3523	4	19294	55	3	0	1	1	1	2	
2125	4	49883	41	1	377	0	0	1	2	

```
In [673]: y_test.head()
```

```
Out[673]:
```

	LOS
716	4
7159	31
2687	4
3523	4
2125	13

```
In [675]: X_test['NEW_LOS'] = np.ceil(X_test.NEW_LOS).astype(int)
```

1271018_CleanDataOnlyFirstVisit

October 28, 2018

```
In [130]: # Importing libraries
import pandas as pd
import numpy as np
import os
import math
import operator
```

```
In [131]: pd.set_option('display.max_columns', 1000)
pd.set_option('display.width', 1000)
```

```
In [132]: os.chdir('A:\\Hackathon\\Data')
```

```
In [134]: df = pd.read_csv('ImputedCleanData.csv')
df.head(10)
```

```
Out[134]:
```

	Unnamed: 0	MEDINCSTQ	SUROPTOT	AGE	CCI	FTMDTF	CM_HTN_C	CM_ALCOHOL	PP086	M
0	0	4	17252	46	3	1	0	0	0	
1	4	2	9467	79	0	0	1	0	1	
2	6	4	24434	73	6	144	0	0	1	
3	7	4	53296	73	7	186	0	0	1	
4	8	3	60571	71	2	50	1	0	1	
5	9	4	8775	86	5	1	1	0	1	
6	10	3	60571	73	2	50	1	0	1	
7	12	4	8113	75	6	168	1	0	0	
8	13	3	8656	52	1	1	1	0	0	
9	14	3	24434	55	6	144	0	0	1	

```
In [135]: df.columns[df.dtypes == 'float64']
```

```
Out[135]: Index(['Unnamed: 88'], dtype='object')
```

```
In [136]: df.columns[df.isna().any()].tolist()
```

```
Out[136]: ['Unnamed: 88']
```

```
In [109]: # df['DISPUNIFORM'].value_counts()
```

```

In [110]: df['CM_AIDS'] = df['CM_AIDS'].fillna(0)
df['CM_ALCOHOL'] = df['CM_ALCOHOL'].fillna(0)
df['CM_DEPRESS'] = df['CM_DEPRESS'].fillna(0)
df['CM_DRUG'] = df['CM_DRUG'].fillna(0)
df['CM_LIVER'] = df['CM_LIVER'].fillna(0)
df['CM_OBESE'] = df['CM_OBESE'].fillna(0)
df['CM_PSYCH'] = df['CM_PSYCH'].fillna(0)
df['DISPUNIFORM'] = df['DISPUNIFORM'].fillna(6)
df['FEMALE'] = df['FEMALE'].fillna(1)
df['GENHOS'] = df['GENHOS'].fillna(1)
df['MEDINCSTQ'] = df['MEDINCSTQ'].fillna(4)
df['PAY1'] = df['PAY1'].fillna(3)
df['PPO86'] = df['PPO86'].fillna(1)
df['RACE'] = df['RACE'].fillna(6)

In [114]: for col in ['CM_AIDS', 'CM_ALCOHOL', 'CM_DEPRESS', 'CM_DRUG', 'CM_LIVER', 'CM_OBESE',
                    'GENHOS', 'MEDINCSTQ', 'PAY1', 'PPO86', 'RACE']:
    df[col] = df[col].astype('int64')

In [125]: df['TOTCHG_X'].describe()

Out[125]: count      16999.0
unique           4.0
top              4.0
freq            5299.0
Name: TOTCHG_X, dtype: float64

In [112]: df['TOTCHG_X'] = df.fillna(df.mean()['TOTCHG_X'])

In [141]: # df = df.drop(['Unnamed: 88'], axis=1)
df = df.drop(['Unnamed: 0'], axis=1)

In [142]: df.head()

Out[142]:
   MEDINCSTQ  SUROPTOT  AGE  CCI  FTMDTF  CM_HTN_C  CM_ALCOHOL  PPO86  MAPP12n  BDTOT
0           4      17252   46    3         1         0           0         0         2      42
1           2       9467   79    0         0         1           0         1         2      40
2           4      24434   73    6       144         0           0         1         2     209
3           4      53296   73    7       186         0           0         1         2     143
4           3      60571   71    2         50         1           0         1         2     208

In [143]: df.shape

Out[143]: (16999, 87)

In [145]: df.to_csv('1271018_CleanData.csv')

In [6]: # Defining a function which calculates euclidean distance between two data points
def euclideanDistance(data1, data2, length):
    distance = 0

```

```

    for x in range(length):
        distance += np.square(data1[x] - data2[x])
    return np.sqrt(distance)

# Defining our KNN model
def knn(trainingSet, testInstance, k):

    distances = {}
    sort = {}

    length = testInstance.shape[1]

    #### Start of STEP 3
    # Calculating euclidean distance between each row of training data and test data
    for x in range(len(trainingSet)):

        #### Start of STEP 3.1
        dist = euclideanDistance(testInstance, trainingSet.iloc[x], length)

        distances[x] = dist[0]
        #### End of STEP 3.1

    #### Start of STEP 3.2
    # Sorting them on the basis of distance
    sorted_d = sorted(distances.items(), key=operator.itemgetter(1))
    #### End of STEP 3.2

    neighbors = []

    #### Start of STEP 3.3
    # Extracting top k neighbors
    for x in range(k):
        neighbors.append(sorted_d[x][0])
    #### End of STEP 3.3
    classVotes = {}

    #### Start of STEP 3.4
    # Calculating the most freq class in the neighbors
    for x in range(len(neighbors)):
        response = trainingSet.iloc[neighbors[x]][-1]

        if response in classVotes:
            classVotes[response] += 1
        else:
            classVotes[response] = 1
    #### End of STEP 3.4

    #### Start of STEP 3.5

```

```
sortedVotes = sorted(classVotes.items(), key=operator.itemgetter(1), reverse=True)
return(sortedVotes[0][0], neighbors)
#### End of STEP 3.5
```